

PrivaMesh - White Paper

A serverless, end-to-end encrypted Web3 messenger on Solana

Version 1.0 - 2026

Abstract

PrivaMesh is a private, serverless messenger with no backend servers of its own. Every piece of shared state lives either on the Solana blockchain or on the user's device. There is no PrivaMesh API, no message relay, no account database, and no operator who can read messages or correlate users.

Messages are end-to-end encrypted with the X3DH key-agreement protocol and the Double Ratchet, and are transported as encrypted memos inside ordinary Solana transactions. Identity and discovery are expressed as on-chain events that any client can independently verify. The result is a communication tool whose privacy and availability do not depend on trusting a company.

Because each account is rooted in a self-custodial keypair, PrivaMesh also ships a lightweight Solana wallet and an on-chain NFT layer (avatars and nicknames) - useful, but secondary to the core product: private messaging.

This paper describes the design goals, the cryptographic protocol, the on-chain transport, the metadata-privacy layer (stealth addresses, cover traffic, fee-paying gas wallets), the serverless discovery mechanism, and the explicit security trade-offs of the architecture. The wallet and NFT layers are covered briefly as secondary features.

1. Motivation

Mainstream messengers protect message *content* with end-to-end encryption but still centralize *everything else*: who talks to whom, when, contact graphs, account identifiers, and the servers that route every packet. That metadata is enough to deanonymize, censor, and coerce.

PrivaMesh starts from a different premise: if there is no server, there is nothing to subpoena, breach, or shut down. The design question becomes "what can be expressed as a verifiable public event, and what must stay on the device?" Content and long-term secrets stay on the device; coordination happens through the blockchain as a neutral, permissionless bulletin board.

2. Design Principles

1. No first-party servers. The app talks only to public Solana RPC endpoints. The vendor operates no message, identity, or storage service.
2. End-to-end by default. Plaintext never leaves the device. Keys never leave the device.
3. Self-custody. A single BIP-39 seed phrase controls funds and on-chain assets. No recovery backdoor.
4. Verifiable, not trusted. Discovery, ownership, and listings are public on-chain events that the client checks cryptographically rather than accepting a server's word.
5. Metadata minimization. Beyond content, the app actively obscures *who*, *when*, and *how much* using stealth addresses, cover traffic, and separable fee payers.
6. Forward secrecy over convenience. Per-message ratcheting means a compromised device cannot decrypt past traffic - at the cost of history not being recoverable from the seed alone (see §12).

3. System Architecture

PrivaMesh is a native iOS (SwiftUI) client. Its only network dependency is the Solana JSON-RPC interface, reached through user-configurable endpoints with automatic rotation on failure.



All durable shared state is one of four kinds of on-chain object:

Concern - On-chain representation

Messages -

Everything else - contacts, decrypted messages, ratchet sessions, private keys - is local to the device and never transmitted.

4. Identity and Key Management

A PrivaMesh account is rooted in a standard BIP-39 mnemonic that derives a Solana ed25519 keypair (the wallet). The public key doubles as the account's network address and message inbox.

Messaging uses a separate set of Curve25519 keys - an identity key, a signed prekey, and ephemeral keys - generated with CryptoKit and stored in the iOS Keychain. These messaging keys are keyed per wallet address, so multiple accounts on one device remain cryptographically unlinkable. They are generated randomly (not derived from the seed); this is what gives PrivaMesh forward secrecy and deniability, and is the reason message history is device-bound (§12).

The seed phrase can optionally be locked behind Face ID / passcode via a Keychain access-control list, without prompting on every transaction.

5. Messaging Protocol

PrivaMesh implements an asynchronous handshake and ratchet.

5.1 Session establishment - X3DH

The first message to a new contact carries a sessionInit envelope:

```
[0x01] [IK_A : 32B] [EK_A : 32B] [EncryptedMessage ?]
```

The sender combines its identity key (IK_A) and a fresh ephemeral key (EK_A) with the recipient's published identity and signed-prekey to compute an X3DH shared secret. Because the envelope is self-contained, a recipient can decrypt a first message from a *stranger* without any prior contact record - enabling unsolicited "message requests" while keeping the protocol stateless on the network.

5.2 Ongoing messages - Double Ratchet

Once a session exists, every message advances a Double Ratchet:

- KDF_RK - HKDF-SHA256(salt = root key, IKM = DH output, info = "PrivaMesh-DR-RK")
- KDF_CK - HMAC-SHA256 chain step
- Encryption - AES-256-GCM, with the 40-byte ratchet header bound as Associated Data

The wire format of an encrypted message is:

```
[0..32) sender ratchet public key
[32..40) message counters (Ns, PN)
[40..] AES-GCM combined (12-byte nonce || ciphertext || 16-byte tag)
```

Each message uses a fresh chain key, so compromising one key never exposes earlier or later messages (forward secrecy + post-compromise security).

5.3 Application payload

Inside the encrypted channel, a typed ChatPayload distinguishes message kinds: text, sol (in-chat payment notice), gift, and cover (decoy - see §6.2). Padding is applied before encryption so ciphertext

length does not reveal message length. The payload also piggybacks the sender's current nickname, NFT avatar seed, and real wallet address, so contacts auto-update without a profile server.

5.4 Transport

The encrypted envelope is base64-encoded and embedded in an SPL Memo instruction (MemoSq4?) within a Solana transaction. The fee payer's signature places the memo on-chain; the recipient discovers it by polling `getSignaturesForAddress` on their inbox address and on per-conversation stealth addresses (§6.1). Message transfers carry zero lamports of value - only the memo - so they never create sub-rent-exempt accounts.

Because the transport is a blockchain memo with a bounded size, PrivaMesh is a text-and-value messenger by design: messages, in-chat SOL payments, and gifts. It deliberately does not transmit photos or large media - keeping every message a small, self-contained on-chain event with no dependency on external file-storage networks.

6. Metadata Privacy

End-to-end encryption hides content. PrivaMesh adds three independent layers to hide the metadata that a blockchain would otherwise expose.

6.1 Stealth addresses

If every message to a contact landed on the same inbox address, the public ledger would reveal the entire conversation graph and frequency. Instead, each conversation derives a one-time address per message from the X3DH shared secret using a deterministic chain (`StealthAddress.address(root, label, index)`). Both parties can compute the next address; an outside observer sees unrelated, single-use addresses with no visible link to either participant. The recipient scans forward along the chain index to collect incoming messages.

6.2 Cover traffic

Conversation *timing* is itself metadata. With cover traffic enabled, the client sends decoy messages (kind = cover) at randomized intervals to real contacts. Decoys are full, valid encrypted messages that advance the ratchet and are silently dropped on receipt - indistinguishable on-chain from real traffic. This breaks timing correlation at the cost of network fees. It is opt-in.

6.3 Gas wallet (separable fee payer)

A Solana transaction's fee payer is public. By default the message tx is paid by the user's main wallet, linking the wallet to the activity. PrivaMesh lets a user designate a separate gas wallet to pay message and cover-traffic fees, funded from an unlinkable source. On-chain, messages then trace to a throwaway fee payer, not the user's identity or balances - the client acts as its own relayer.

6.4 Disappearing messages

Per-contact, the client can auto-purge decrypted copies after a chosen interval. This is a *local* control over plaintext retention; the on-chain ciphertext is immutable and remains (undecryptable to anyone without the ratchet state).

7. Serverless Discovery

To message someone by a human-readable nickname without a directory server, PrivaMesh uses an on-chain registry. A user publishes a prekey bundle - the keys needed to start an X3DH session - as a memo to a well-known registry address, indexed by nickname.

The critical anti-MITM property: the published bundle is wallet-signed. The ed25519 signature ties the messaging keys to the Solana address that posted them, so a third party cannot inject a forged bundle for a nickname they do not control. Clients verify the signature before trusting any discovered bundle. This replaces a trusted key server with a publicly auditable, self-authenticating record.

8. Wallet and Payments

PrivaMesh is a full self-custodial Solana wallet: send/receive SOL, view balance and transaction history (read directly from RPC), and scan/show addresses by QR. In-chat payments and "gifts" are real on-chain SOL transfers announced through the encrypted channel, so the recipient gets both the value and a private notification. All fees are real network fees; the vendor takes no cut of peer-to-peer transfers.

9. NFT Marketplace

9.1 On-chain ownership

Avatars and nicknames are minted as genuine SPL tokens with supply 1 and 0 decimals - NFTs in the structural sense - each carrying a `PNFT1:<designId>` memo in its mint transaction. Ownership is therefore seed-portable: on a new device, the client reconstructs the collection by reading the wallet's token accounts (`getTokenAccountsByOwner`) and resolving each mint's design memo. No PrivaMesh database records who owns what.

9.2 Serverless listings

Market listings are likewise memo events posted to a market registry address. Any client folds the current order book by replaying `getSignaturesForAddress` over the registry - list, delist, and sale events reduce to live state with no marketplace server. Primary sales pay the seller (or, for genesis items, the project) directly in SOL, with a transparent on-chain developer commission. Reserved system handles are excluded from sale and minting.

10. Premium Tier (PrivaMesh+)

A subscription unlocks creator and power-user features: minting custom NFT nicknames (up to three), unlimited simultaneous sale listings (free accounts are capped), a verification checkmark on one's own profile, and up to three independent accounts on the device (free accounts get one). The subscription itself is the one component that uses a conventional rail - Apple StoreKit 2 in-app purchase - because

App Store policy requires it for digital goods. Marketplace value transfer remains on-chain in SOL.

11. Multi-Account

A device can hold several fully independent accounts, each with its own seed, balance, nickname, chats, and messaging identity. Because messaging keys are keyed per wallet address, accounts are mutually unlinkable; switching accounts re-scopes the entire app. Accounts can be created fresh or imported by seed phrase.

12. Security Model and Trade-offs

What an adversary cannot do. Without the device's keys, no party - including the vendor and any RPC operator - can read message content, forge a discovered identity (signatures are verified), or seize funds. There is no central store to breach.

What remains visible. The blockchain is public. Transaction existence, timing (mitigated by cover traffic), and fee payers (mitigated by the gas wallet) are observable to a global passive adversary. Stealth addresses unlink the recipient graph but do not hide that *some* transaction occurred. Anonymity therefore depends on using the privacy layers, and on how the funding wallets were sourced.

Forward secrecy vs. recoverability. Messaging keys and ratchet sessions are random and device-local. This is what delivers forward secrecy and deniability - and it means that restoring an account from its seed phrase on a new device recovers funds and NFTs, but not contacts or message history. On-chain ciphertext exists but is undecryptable without the original ratchet state. This is a deliberate trade-off, matching the behavior of comparable secure messengers; an optional encrypted backup would reverse it at the cost of weaker forward secrecy.

Local device security. Since the device holds everything, the seed can be locked behind biometrics/passcode, and a full wipe erases all seeds, per-account messaging identities, contacts, and chats.

Known limitations. Marketplace uniqueness and some ownership checks are enforced client-side; RPC endpoints can observe a user's read queries (mitigated by rotation and the option to self-host); and availability depends on public Solana infrastructure.

13. Conclusion

PrivaMesh demonstrates that a private messenger can run with zero servers - and that a self-custodial wallet and an on-chain NFT layer can ride along for free as secondary features. By expressing coordination as verifiable on-chain events and keeping content and secrets on the device, it removes the operator from the trust model entirely: there is no one to ask for your messages, because no one has them. The cost of that posture - history that is bound to the device, and metadata that must be actively obscured rather than hidden by a trusted middleman - is made explicit rather than hidden. For users who would rather trust mathematics and a public ledger than a company, that is the right trade.

PrivaMesh runs on Solana mainnet-beta. Network actions cost real SOL fees. This document describes the system as implemented and is not investment advice.